



LLMs Meet Isolation Kernel: Lightweight, Learning-free Binary Embeddings for Fast Retrieval

Zhibo Zhang Yang Xu Kai Ming Ting Cam-Tu Nguyen

National Key Laboratory for Novel Software Technology

Nanjing University

zhibozhang@smail.nju.edu.cn, xuyang@lamda.nju.edu.cn, {tingkm, ncamtu}@nju.edu.cn

To Appear in ACL 2026

IKE

This paper presents **Isolation Kernel Embedding (IKE)**, a learning-free vector compression method designed for high-dimensional dense retrieval. It leverages the Isolation Kernel (IK) to transform embeddings from large language models (LLMs) into binary codes. Through compressed representations and efficient bitwise computation, IKE significantly reduces memory overhead and retrieval latency, while maintaining performance comparable to the LLM embedding space.

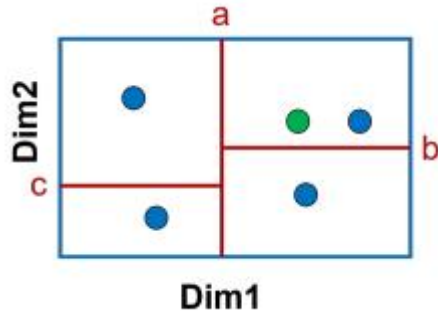
- Experiments on multiple text retrieval datasets demonstrate that IKE offers up to 16.7× faster retrieval and 16× lower memory usage than LLM embeddings, while **maintaining comparable or better accuracy**.
 - In **ANN** (Approximate Nearest Neighbor) Search, pairing IKE with an HNSW index achieves up to 5x higher throughput than operating in the LLM space;
 - **Further comparison** with **learning-based method (CSR)** and **other (no-learning) compression methods (hashing and quantization)** demonstrate the advantages of IKE.
-

IKE Isolation Kernel

Isolation Kernel is a data-dependent kernel that computes the similarity **based on the partitions** in the data space.

Construct partition from data subset

Given a data set $D = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$, we define $\mathcal{D} \subset D$ as a subset of ψ data points, each drawn from D with equal probability $\frac{1}{n}$. A **partition** H is derived so that each **isolating partition** $\theta \in H$ isolates a point from the rest of $\mathcal{D}$.



IK computation

Let $\mathcal{H}_\psi(D)$ denote **the set of all partitions** that are admissible under the data set D .

IK of any two points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ is defined to be the **expectation** taken over the probability distribution $P_{\mathcal{H}_\psi(D)}$ on all partitions $H \in \mathcal{H}_\psi(D)$ that **both $\mathbf{x}$ and $\mathbf{y}$ fall into the same isolating partition** $\theta \in H$:

$$K_{\psi,D}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{H \sim P_{\mathcal{H}_\psi(D)}} [\mathbb{I}(\mathbf{x}, \mathbf{y} \in \theta | \theta \in H)]$$

In practice, IK is approximated using t partitions, each derived from an **independently sampled data subset** $\mathcal{D}_i \subset D, i \in [1, \dots, t]$:

$$K_\psi(\mathbf{x}, \mathbf{y}|D) \simeq \frac{1}{t} \sum_{i=1}^t \mathbb{I}(\mathbf{x}, \mathbf{y} \in \theta | \theta \in H_i)$$

IKE Mapping Function

$$K_{\psi}(\mathbf{x}, \mathbf{y}|D) \simeq \frac{1}{t} \sum_{i=1}^t \mathbb{I}(\mathbf{x}, \mathbf{y} \in \theta | \theta \in H_i)$$

Since each partition H_i consists of ψ isolating partitions, and each data point falls into exactly one isolating partition, H_i can be viewed as a mapping function that maps a data point to a one-hot encoding of length ψ , indicating the region into which the data point falls:

$$H_i(\mathbf{x}) = \{0, 1\}^{\psi}$$

Aggregating the mappings from the t partitions, we can define a mapping: $\Phi: \mathbf{x} \mapsto \{0, 1\}^{t \times \psi}$, which maps a data point to a binary representation of length $t \times \psi$.

$$\Phi(\mathbf{x}) = \left[\overbrace{[0, 0, \dots, 1, \dots, 0, 0]}^{\text{partitions } t}, \dots, \dots \right]$$

$\underbrace{\hspace{10em}}_{\psi \text{ numbers}}$

Isolation Kernel can be approximated by the normalized inner product:

$$K_{\psi}(\mathbf{x}, \mathbf{y}|D) \simeq \frac{1}{t} \langle \Phi(\mathbf{x}|D), \Phi(\mathbf{y}|D) \rangle$$

IKE Representing compression

$$\Phi(\mathbf{x}) = [0, 0, \dots, 1, \dots, 0, 0, \dots, \dots]$$

Given the high sparsity of $\Phi(\mathbf{x})$, we can compress it by storing only the indices of the “1”s, yielding a t -dimensional IKE representation:

$$\Phi^{idx}(\mathbf{x}) \in \{1, \dots, \psi\}^t$$

Isolation Kernel then can be calculated based on comparison operation:

$$K_{\psi}(\mathbf{x}, \mathbf{y}|D) \simeq \frac{1}{t} \sum_{i=1}^t \mathbb{I}(\Phi_i^{idx}(\mathbf{x}) = \Phi_i^{idx}(\mathbf{y}))$$

Compression ratio of $\Phi^{idx}(\mathbf{x})$

Each element is stored using $\lceil \log(\psi) \rceil$ bits. Let $\mathbf{x}$ be a 4096-dimensional FP32 vector; the **compression ratio** can then be calculated as follows:

$$t = 4096, \psi = 16$$

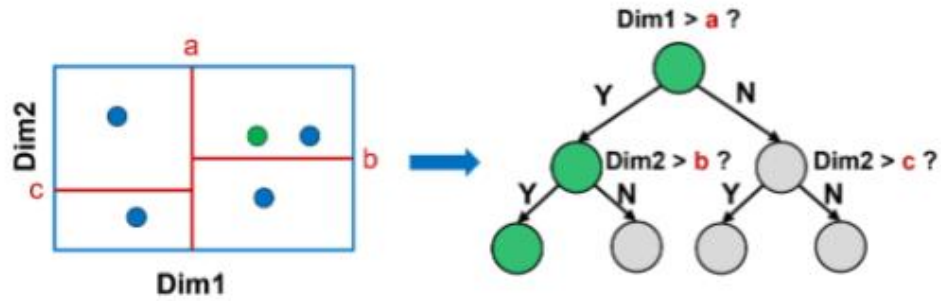
$$r = 1 - \frac{4096 \times \lceil \log(16) \rceil}{4096 \times 32} = 87.5\%$$

$$t = 4096, \psi = 2$$

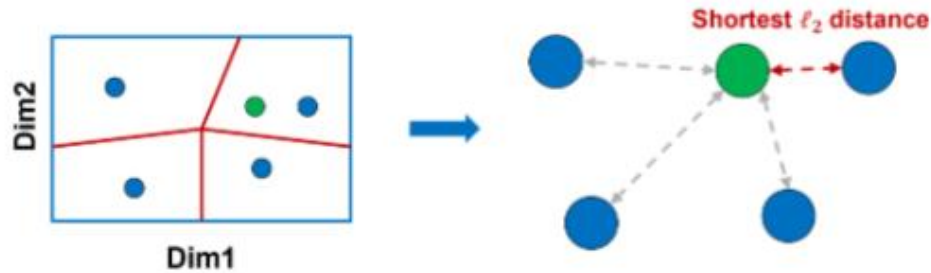
$$r = 1 - \frac{4096 \times \lceil \log(2) \rceil}{4096 \times 32} = 96.875\%$$

Bitwise Similarity Calculation

IKE IK implementations



(a) iForest: a , b , c are splitting values for dimensions Dim1 and Dim2.



(b) Voronoi Diagram (VD): red lines indicate splitting hyper-planes, that split the space into VC cells.

Two commonly used implementations are Isolation Forest (iForest) and Voronoi Diagram (VD), both of which are learning-free.

iForest

iForest is a feature-granularity partitioning method: a child node is obtained by randomly selecting a feature q and a split value p uniformly from the interval between the minimum and maximum values of feature q .

VD

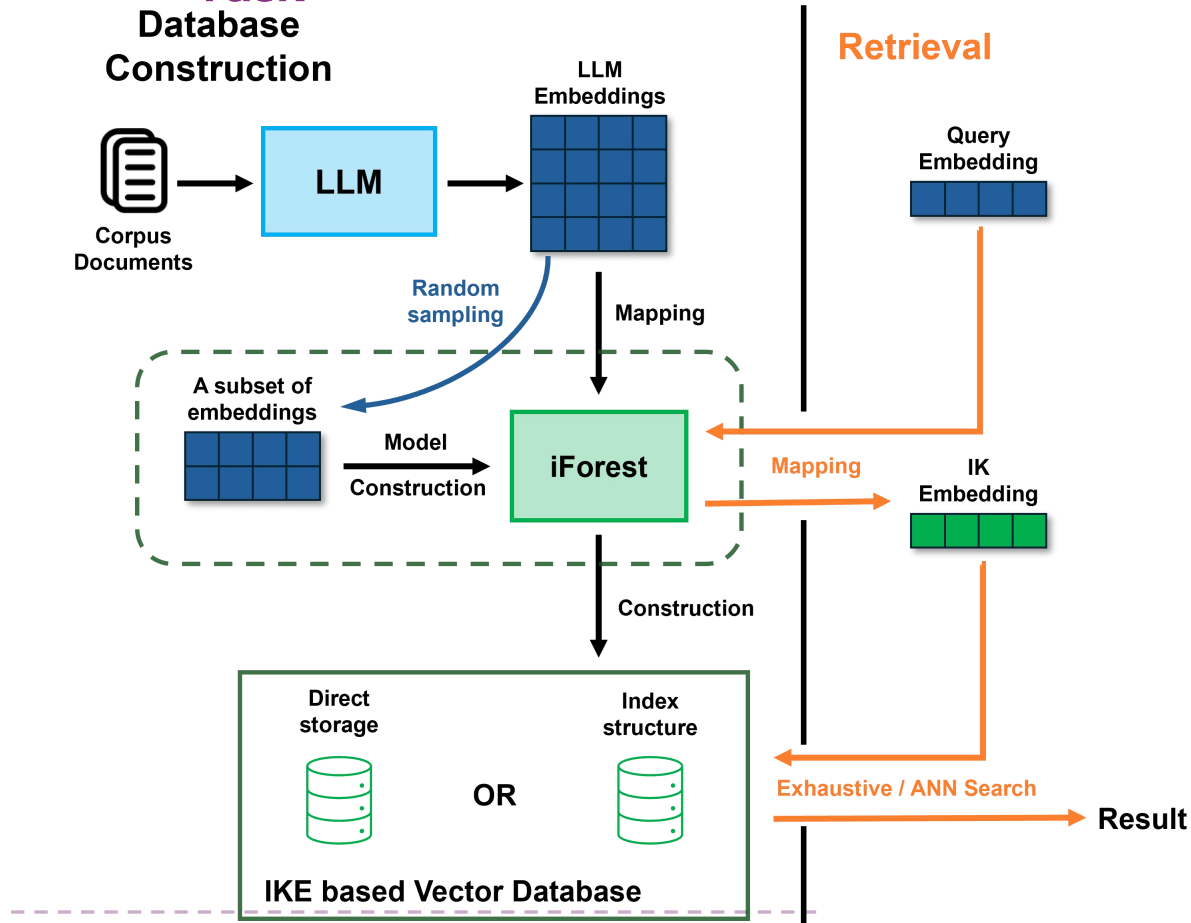
VD is an instance-granularity partitioning method: a mapped point selects the point in the subset $\mathcal{D}$ closest to it in terms of ℓ_2 distance as its isolating partition.

IKE adopts **iForest** as the default partitioning method.

Methodology



IKE Workflow of IKE in the Text Retrieval Task



1. All documents in the corpus are first converted into **embedding vectors** using a large language model.
2. From the generated embeddings, a **random subset is sampled to construct an iForest model**—a process that requires no training.
3. Based on this iForest model, all embeddings are then mapped into the IKE representation space, building an **IKE-based vector database**.
4. As a representation method, IKE is not only suitable for **exhaustive search** scenarios but can also be combined with **various indexing methods** to support ANN search.
5. During retrieval, a given query embedding is first transformed into the IKE space, after which the corresponding search operation is performed in the database.

Methodology



IKE VDeH¹ and IKE_{VD}

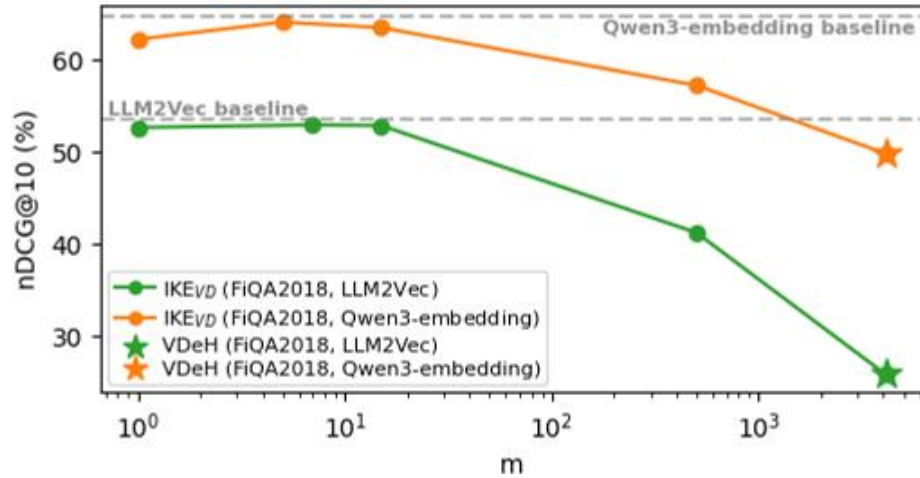
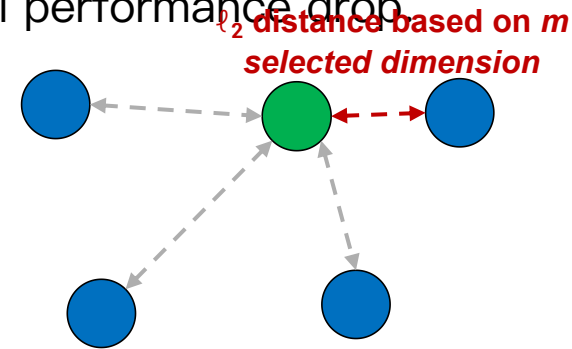


Figure 2: Effect of parameter m in IKE_{VD} ($t=4096$) on nDCG@10 across two LLM embedding datasets. The pentagram marker indicates the performance of IKE_{VD} when $m = d$, which is equivalent to the VDeH method.

¹Voronoi diagram encoded hashing

We also investigated the performance of VD partition on text retrieval tasks (known as VDeH¹ method). As shown in the left figure, compared to the original embedding representation, VD yields a substantial performance drop.



Inspired by the granularity-based partitioning approach in IKE, we propose IKE_{VD} , which considers only m randomly sampled dimensions when partitioning the space, rather than all d dimensions. It can be observed that when m is small (< 10), high retrieval accuracy is achieved, and as m increases, the accuracy gradually declines.

Some theoretical analysis from Partition Diversity in Appendix C.4.

Experiments



IKE

Datasets

Dataset	Relevance	Corpus Size	Query Size
<i>MTEB</i>			
HotpotQA	Binary	5,233,329	7,405
FiQA2018	Binary	57,638	648
FEVER-HN	Binary	163,698	1,000
Touche2020.V3	3-Level	303,732	49
<i>Extra Datasets</i>			
Istella22 (subset)	5-Level	1,000,000	234
TREC DL 23 (subset)	4-Level	1,000,000	82

Backbones

- Qwen3–embedding–8B
- LLM2Vec (Mistral-7B backbone)

Both producing 4096-dimensional embeddings.

Hyperparameter Tuning

The parameter ψ is determined for each dataset through a grid search on a validation set. Specifically, we tune over the range [2,16] and select the best value.

Experiments



IKE Comparison with LLM embeddings – Exhaustive Search

LLM2Vec

Dataset	Space Type	Space	Time (s)		MRR@10	nDCG@10
			Other	Search		
HotpotQA	LLM		46.33	326.61	83.83	74.09
	IKE ($\psi=12$)	$\downarrow 8\times$	18.17	54.82 ($\downarrow 6.0\times$)	83.69 ± 0.13	73.24 ± 0.09
FiQA2018	LLM		0.29	0.47	60.33	53.49
	IKE ($\psi=6$)	$\downarrow 8\times$	0.20	0.063 ($\downarrow 7.5\times$)	59.97 ± 0.32	52.59 ± 0.31
FEVER-HN	LLM		0.84	1.10	91.52	90.45
	IKE ($\psi=15$)	$\downarrow 8\times$	0.59	0.26 ($\downarrow 4.2\times$)	91.36 ± 0.23	90.27 ± 0.16
Istella22	LLM		5.05	1.56	68.11	63.88
	IKE ($\psi=16$)	$\downarrow 8\times$	3.26	0.62 ($\downarrow 2.5\times$)	68.92 ± 0.88	63.86 ± 0.50
TREC DL 23	LLM		5.11	0.90	97.56	73.10
	IKE ($\psi=3$)	$\downarrow 16\times$	2.23	0.16 ($\downarrow 5.6\times$)	98.11 ± 1.02	71.78 ± 0.62
Touche2020.V3	LLM		1.57	1.44	82.65	51.61
	IKE ($\psi=7$)	$\downarrow 8\times$	1.06	0.095 ($\downarrow 15.2\times$)	82.37 ± 3.33	51.02 ± 1.18

Qwen3-embedding

Dataset	Space Type	Space	Time (s)		MRR@10	nDCG@10
			Other	Search		
HotpotQA	LLM		41.70	332.66	89.76	76.85
	IKE ($\psi=10$)	$\downarrow 8\times$	14.51	57.21 ($\downarrow 5.8\times$)	88.92 ± 0.13	75.41 ± 0.10
FiQA2018	LLM		0.28	0.46	72.05	64.66
	IKE ($\psi=15$)	$\downarrow 8\times$	0.24	0.069 ($\downarrow 6.7\times$)	70.95 ± 0.37	63.17 ± 0.31
FEVER-HN	LLM		0.88	1.22	93.91	92.50
	IKE ($\psi=13$)	$\downarrow 8\times$	0.39	0.29 ($\downarrow 4.2\times$)	93.59 ± 0.22	92.09 ± 0.19
Istella22	LLM		5.23	1.62	65.25	59.44
	IKE ($\psi=11$)	$\downarrow 8\times$	1.64	0.61 ($\downarrow 2.7\times$)	65.11 ± 0.70	59.63 ± 0.55
TREC DL 23	LLM		5.08	0.84	95.73	73.79
	IKE ($\psi=6$)	$\downarrow 8\times$	2.91	0.32 ($\downarrow 2.6\times$)	96.93 ± 0.42	72.78 ± 0.28
Touche2020.V3	LLM		1.55	1.59	96.94	76.41
	IKE ($\psi=16$)	$\downarrow 8\times$	0.47	0.095 ($\downarrow 16.7\times$)	95.83 ± 1.41	73.35 ± 1.13

Experiments

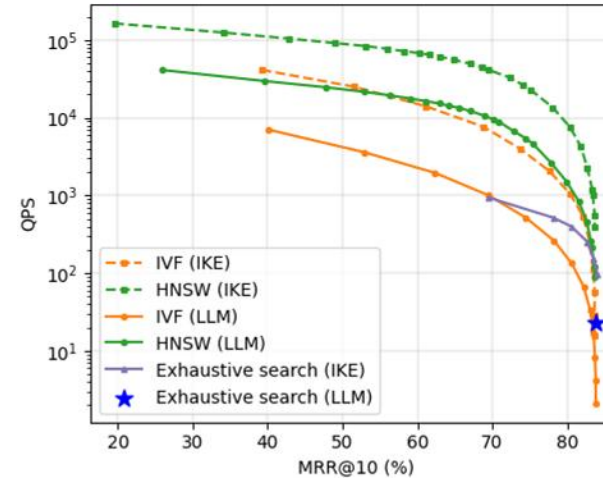


IKE Comparison with LLM embeddings – ANN

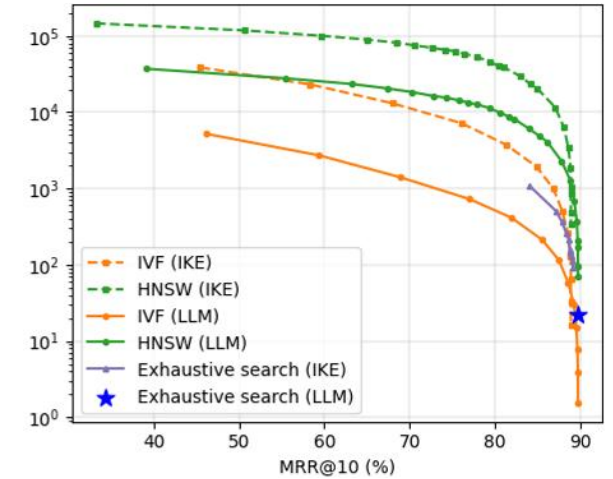
As an embedding representation method, IKE can be integrated with various indexing methods. In our experiments, we employ two widely used indexes: HNSW and IVF

IVF (IKE) We use k -means clustering to partition the data and construct the IVF index in the LLM embedding space. During retrieval, the query's LLM embedding is used to identify the closest clusters, and then the similarity between the query and the data points within these clusters is computed in the IKE space.

HNSW (IKE) The index construction and search processes follow the methodology proposed in the original paper, with the only modification being the replacement of the cosine similarity defined in the LLM embedding space with the similarity defined in the IKE space.



(a) HotpotQA (LLM2Vec)



(b) HotpotQA (Qwen3)

Experiments



IKE Comparison with Compression Methods – Learning-based and Learning-free

Under the same **fixed code length**, we compare it with both learning-based and learning-free methods.

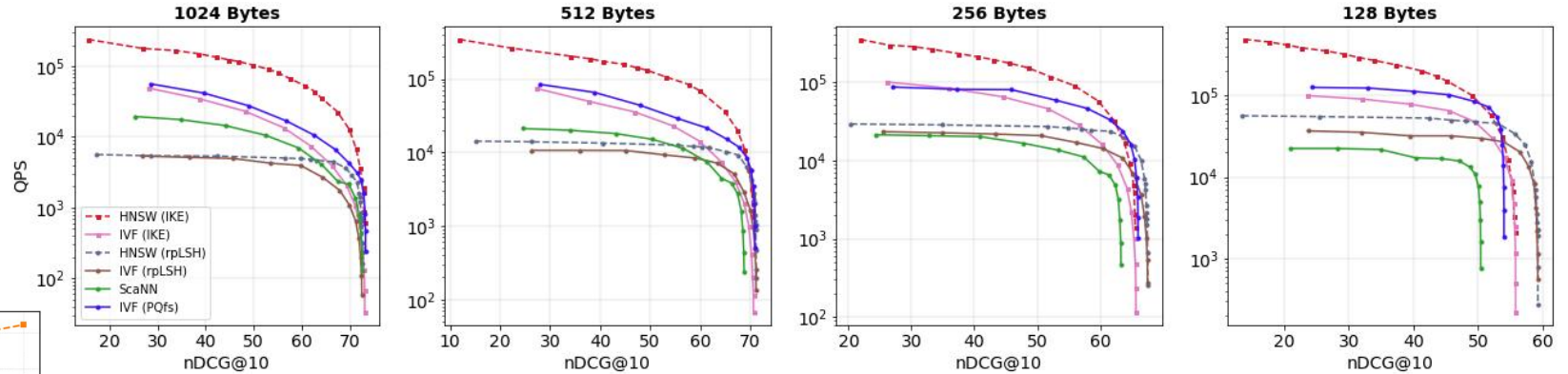


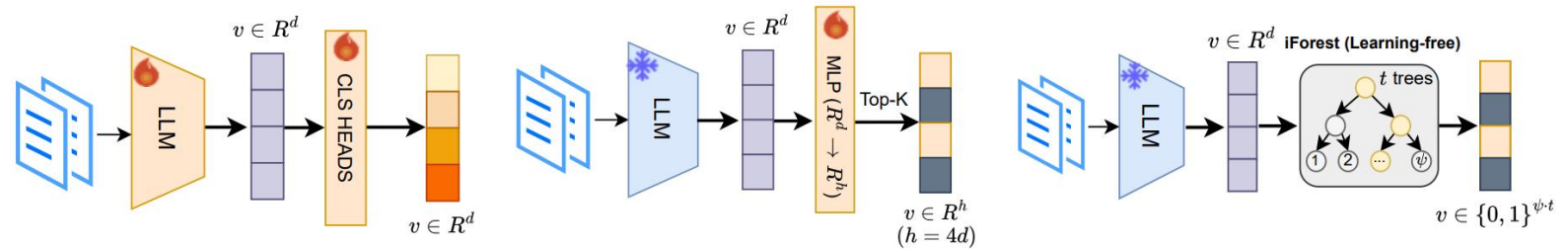
Figure 6: Performance Comparison of Compression Methods on ANN Retrieval: QPS vs. nDCG@10 on the HotpotQA (LLM2Vec) dataset. The curves are obtained by adjusting the search parameter of the index structures.

🔥 Trainable Module ❄️ Frozen Module ■ Zero Value ■ Non-Zero Value

a) Search Time

b) Accuracy

Figure 5: Comparison of Retrieval Accuracy and Search Time between CSR and IKE in the exhaustive search setting on the HotpotQA (LLM2Vec) dataset.

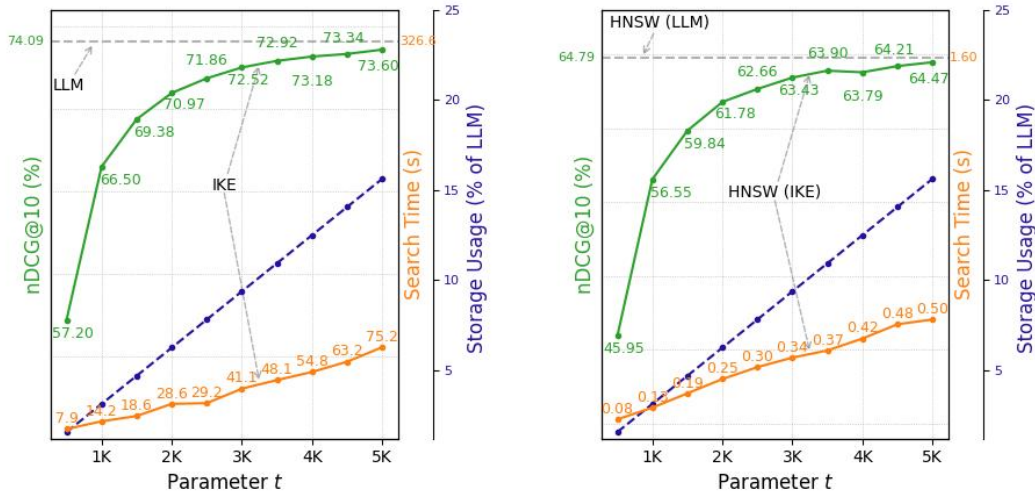


(a) MRL (Learning-based)

(b) CSR (Learning-based)

(c) IKE (No Learning)

IKE



(a) Exhaustive Search
(HotpotQA, LLM2Vec)

(b) HNSW
(HotpotQA, LLM2Vec)

Effect of parameter t

Retrieval performance improves as t increases

t	ψ	Other time (s)	Search time (s)	MRR@10 (%)	nDCG@10 (%)
4096	2	1.18	0.16	91.06	75.71
4096	4	1.69	0.25	90.99	75.83
4096	8	2.49	0.35	90.94	76.02
4096	16	3.70	0.36	91.43	76.35

Effect of parameter ψ

Retrieval performance does not necessary improve as ψ increases.

Note that even with small values of ψ , IKE maintains remarkably high retrieval accuracy while reducing both mapping overhead and query latency.

Evolution from VDeH to IKE

- **The Bottleneck of VDeH on LLMs:**

- LLM embedding spaces have different characteristics to those of typical text databases.
- Using all dimensions restricts partition diversity.
- Commonly known hashing criteria (coverage, entropy, independence) are insufficient.

- **The Power of iForest (IKE):**

- Randomized splitting generates highly diverse base partitioners.
- Ensembling thousands of independent trees cancels out individual biases.

- **Key Insight: The 4th Criterion**

- **Diverse Partitioning:** A critical necessity for robust binary representations.
 - *Ensemble:* Simple, diverse learners outperform complex, correlated ones.
-

While our method demonstrates strong performance on the text retrieval task, its effectiveness on cross-modal retrieval tasks (e.g., Image-to-Text or Text-to-Image) remains limited.

This may be attributed to the modality between textual and visual representations, which poses challenge for cross-modal alignment. This aspect has not been investigated in depth in this paper and is left for future research.

What have you learned today?

1. What are the key differences between IKE and VDeH?
2. The LLM embedded databases used have ground-truths for each query. Is this necessary for retrieval evaluation?
3. What are the advantages of using IKE on an LLM embedded database? How is this achieved?
4. IKE, without learning, outperforms the state-of-the-art deep-learning-based method CSR (ICML-2025). Discuss the potential reasons why.